

NutriGPT

Project Description:

Welcome to **NutriGPT** – your personalized nutrition coach and AI-powered health assistant! Whether you are tracking daily calories, monitoring macronutrients, searching for foods semantically, or consulting an AI assistant about your diet, NutriGPT makes managing your nutrition simple, data-driven, and personal.

With NutriGPT, users can register, build detailed profiles containing weight, height, age, activity level, dietary preferences, and allergies, and get automatically calculated personalized daily calorie and macro goals (protein, carbs, and fat). Built on the powerful **MERN Stack (MongoDB, Express.js, React, Node.js)**, the application delivers a smooth, modern, and responsive health-tracking experience. Leveraging a Retrieval-Augmented Generation (RAG) pipeline powered by **Gemini AI (gemini-2.5-flash** for conversational coaching and **gemini-embedding-001** for vector embeddings), NutriGPT offers highly personalized dietary guidance grounded directly in the user's real profile and daily meal log.

NutriGPT brings intelligent nutrition coaching to your fingertips.

Scenario-Based Case Study:

Sarah, a fitness enthusiast, wants to lose weight and optimize her protein intake. She finds tracking meals tedious because traditional apps rely on exact keyword matching for food searches (e.g., searching for "egg" doesn't show "scrambled whites" unless she scrolls forever). She also struggles to understand how to adjust her diet when she falls short of her daily protein target, and lacks access to personalized nutrition advice that is aware of what she actually ate earlier in the day.

The Solution:

NutriGPT provides an engaging and comprehensive health-tracking ecosystem designed to help users like Sarah reach their nutrition goals through AI-powered search and real-time grounding.

How NutriGPT Helps Users

1. User Registration & Authentication

- Create a secure account, manage your credentials, and enjoy personalized dashboard access.

2. Personalized Profile Setup & Targets

- Input age, weight, height, gender, activity level, and goals (weight loss, muscle gain, maintenance) to automatically calculate daily calorie and macro targets using the Mifflin-St Jeor equation.

3. Semantic Food Search & Custom Food Logging

- Search the food database semantically (e.g., searching "healthy green side" returns "spinach salad" even without exact keyword matches) using vector embeddings, or add custom foods.

4. Daily Meal Logging

- Log food items under breakfast, lunch, dinner, or snack categories, and track progress relative to daily calorie and macro targets.

5. AI Nutrition Coach Chat (RAG)

- Chat with an AI assistant that dynamically pulls context from your profile metrics and daily food logs to provide highly personalized advice.

6. Progress Tracking & Analytics Dashboard

- Access progress visualizations (charts of weight, target vs. consumed calories) to monitor your health journey over time.

System Requirements:

To ensure smooth development, deployment, and usage of the NutriGPT Web Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable application.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v16 or above):** Provides the runtime environment for building and running the backend server and managing dependencies. Download Node.js
- **npm (v8 or above):** Package manager to install dependencies for frontend and backend libraries.
- **React.js (v18):** Frontend library used to build the responsive Single Page Application (SPA).
- **Vite:** Modern frontend build tool supplying fast Hot Module Replacement (HMR).
- **Browser:** Google Chrome, Firefox, or Microsoft Edge (latest versions) — For rendering and debugging the UI.
- **Express.js:** Backend framework for building RESTful APIs.
- **MongoDB:** NoSQL database (Local or Atlas) to store users, custom foods, meal logs, chat histories, and activity audits. Download MongoDB
- **Gemini API Key:** Free or commercial key from Google AI Studio to power vector embeddings and chat features. Get Gemini Key
- **Postman / Insomnia:** Tools for testing RESTful API endpoints during development.
- **Visual Studio Code:** Preferred lightweight code editor with Terminal support and extension ecosystem.
- **Git & GitHub:** Version control system to manage source code and pull requests.

2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

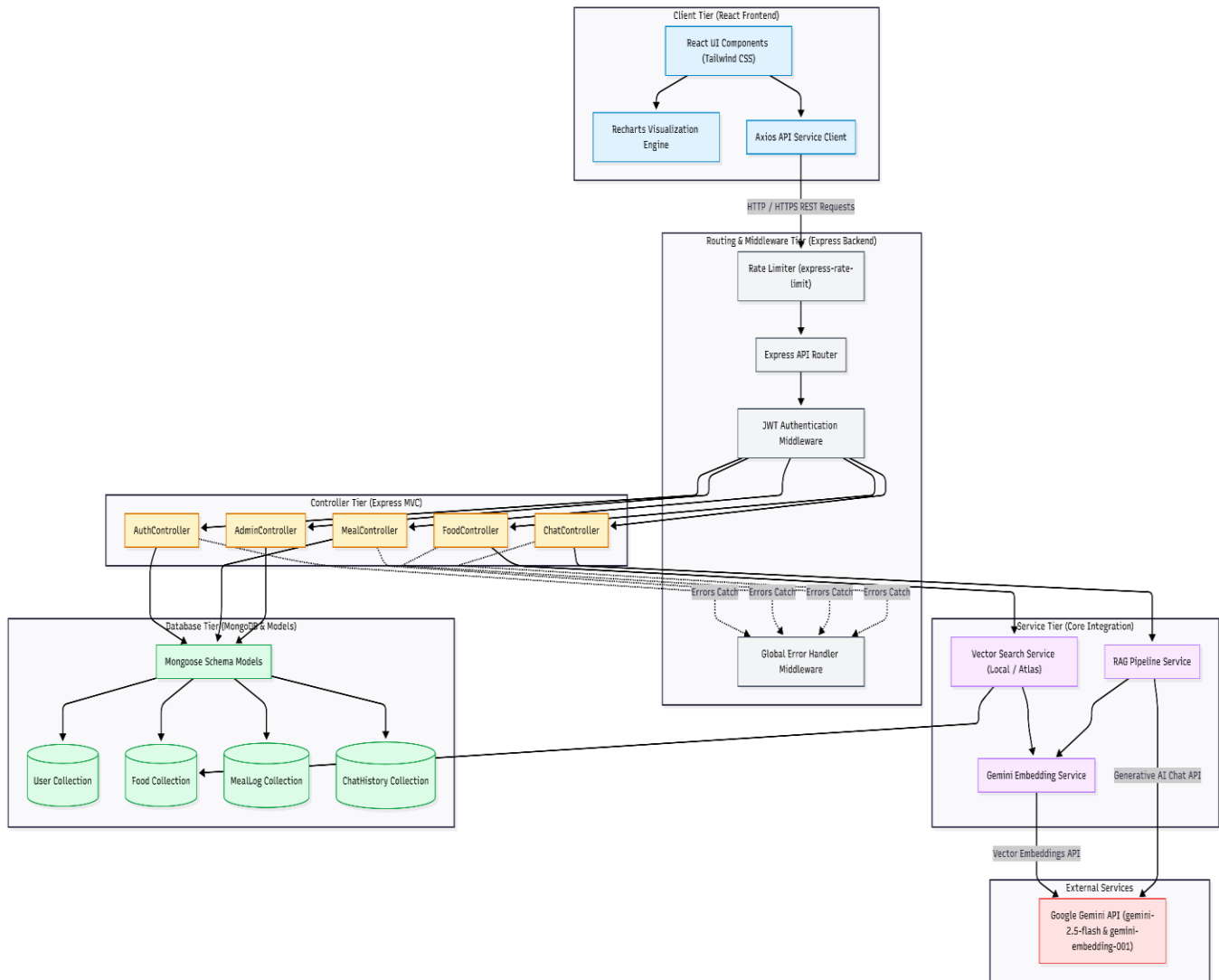
- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — For fast compilation and multitasking.
- **RAM:** Minimum 8 GB (16 GB recommended) — To run the backend server, frontend development client, IDE, and multiple browser tabs simultaneously.
- **Storage:** At least 1 GB free space — For packages, local MongoDB databases, and development files.
- **Display:** 1920x1080 resolution — Recommended for comfortable split-screen coding and UI design validation.

Project Architecture:

NutriGPT follows a modular full-stack architecture using the MERN stack:

`Frontend (React + Vite) <==> Backend (Express + Node.js) <==> Database (MongoDB) + AI (Gemini API)`

Technical Architecture:



The **NutriGPT** application is designed with a secure and modular technical architecture to ensure high performance, reliable food search, and context-aware chat interactions.

1. User Interface (UI)

The frontend is a modern web application built using React 18, Vite, Tailwind CSS, and Recharts. The user interface provides responsive dashboard analytics and forms:

- **Dashboard Page:** Displays progress charts, remaining calorie rings, logged meal summaries, and macro breakdowns.
- **Profile Setup & Management:** Dynamic forms to enter weight, height, activity level, diet style, and allergies, triggering automatic target recalculation.
- **Meals Log:** Add meals under breakfast, lunch, dinner, or snacks, choosing foods from search results and inputting servings.
- **AI Chat:** Interactive message panel with conversational context and RAG details.
- **Admin Dashboard:** Access-controlled interface to view all users, manage database foods, and monitor server operations.

2. Web Server

The backend is built on Node.js and Express.js using the MVC pattern. It handles API requests, routes, token verification, and interfaces with the database and AI services.

3. API Gateway / Router

Express routing layers map API routes `/api/auth`, `/api/foods`, `/api/meals`, `/api/chat`, and `/api/admin` to their respective controller functions.

4. Authentication Service

Handles secure login, registration, and role authorization using JWT (JSON Web Tokens) and bcrypt. Middlewares inspect requests for valid user roles (`user` or `admin`) before granting access to protected routes.

5. Database (MongoDB)

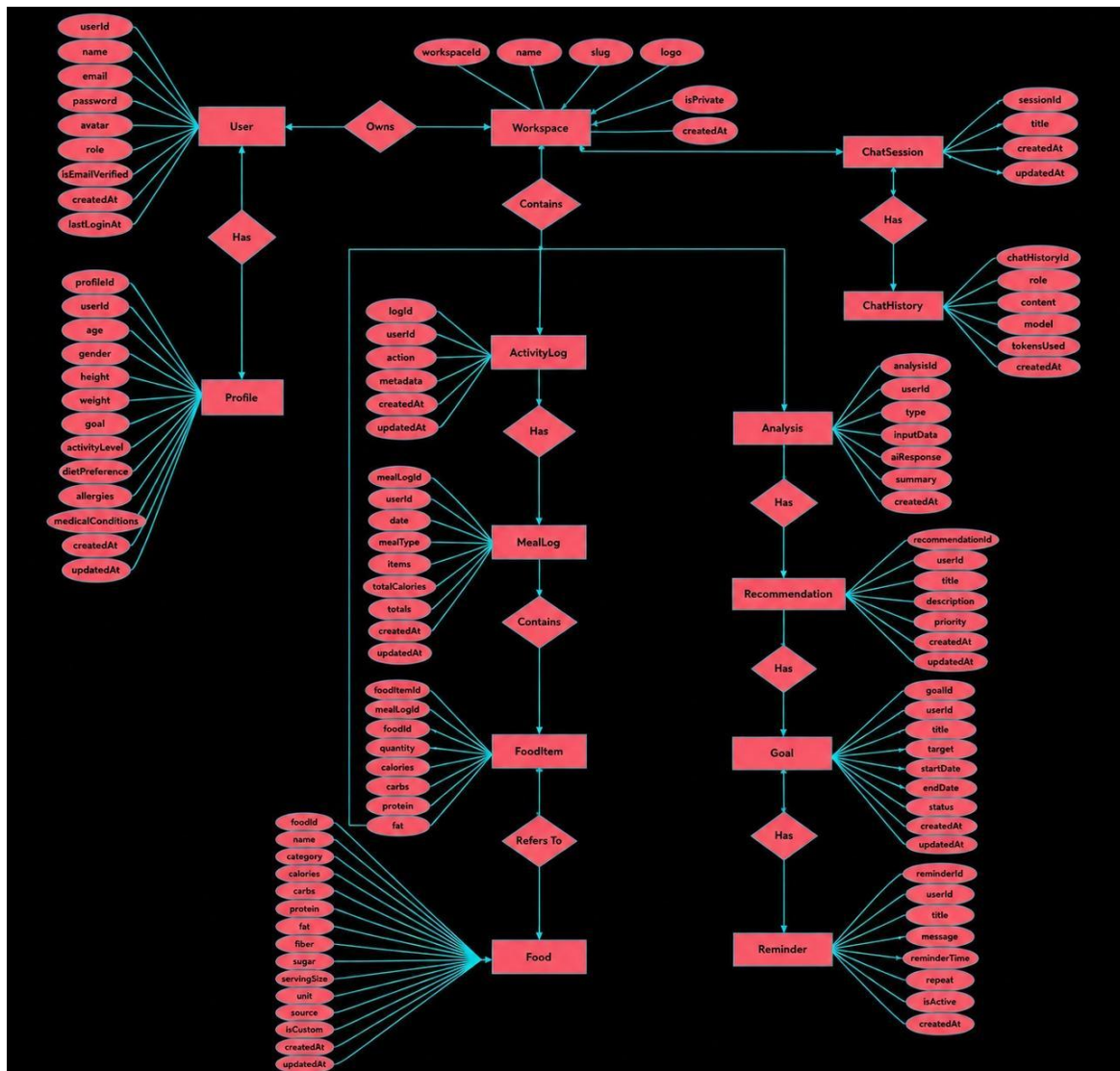
Mongoose schemas structure application data:

- **Users:** Credentials, health metrics, daily target macro bounds, and role.
- **Foods:** Calorie values, nutrient breakdowns, diet tags (keto, vegan, etc.), and vector embeddings.
- **MealLogs:** Groupings of food items consumed by a user on a specific date.
- **ChatHistories:** Multi-turn message transcripts between the user and assistant.
- **ActivityLogs:** System audit logs for tracking activity history.

6. AI and RAG Pipeline Service

Interfaces with Gemini API. Embeds search phrases via `gemini-embedding-001` to carry out semantic searches. When chatting, the service fetches current health profile data and daily meal logs, constructing a structured prompt for `gemini-2.5-flash` to generate personalized nutrition coaching.

ER Diagram:



Here are the relationships between collections in the NutriGPT database:

1. User and MealLog Relationship

- **Type:** One-to-Many
- **Description:** A single User can log multiple meals across different days.

- **Implementation:** The **MealLog** schema stores a reference to the **User ID** (**user** field) and lists the logged items.

2. User and ChatHistory Relationship

- **Type:** One-to-Many
- **Description:** A single User can maintain multiple chat history sessions.
- **Implementation:** The **ChatHistory** schema contains a **user** field storing the user's **ObjectId**.

3. User and ActivityLog Relationship

- **Type:** One-to-Many
- **Description:** A single User can trigger multiple activity log events (registration, login, profile updates).
- **Implementation:** Stored in the **ActivityLog** schema referencing the user's **ObjectId**.

4. Food and MealLog.items Relationship

- **Type:** One-to-Many
- **Description:** A Food item defined in the global database can appear in multiple meal log entries.
- **Implementation:** The nested **items** array inside the **MealLog** schema stores the **Food** reference ID (**food** field) and denormalizes the name and macro values for fast retrieval.

Features

- **Role-Based Access Control (RBAC):** Users are partitioned into standard **user** and **admin** roles, guarding admin actions such as managing all registered accounts and modifying food entries.
- **Secure User Authentication:** Secure account setup using email/password validation, salted password hashing via bcrypt, and stateless sessions utilizing JWTs.
- **Profile Setup & Target Engine:** Collects physical stats and goal preferences, utilizing the Mifflin-St Jeor equation to estimate BMR, TDEE, and daily macro allocations (30% Protein, 40% Carbs, 30% Fat).
- **Semantic Vector Search:** Integrates local cosine-similarity vector searches over food embeddings, allowing users to search by category or synonym without exact match strings. Supports switching to Atlas Vector Search in production.
- **Interactive Dashboard Analytics:** Visualizes daily calorie targets versus consumption, tracks macro splits in real-time, and graphs weight changes using Recharts.
- **Dynamic Daily Meal Logging:** Quick addition of standard database items or custom manual foods to daily intake categories (breakfast, lunch, dinner, snack).

- **AI Chat Assistant (RAG Pipeline):** Natural-language conversational interface grounded in the user's profile information and daily calorie intake records.
- **Tailwind Dark/Light Mode Theme support:** Implemented theme context providing toggleable light/dark modes for accessibility.

Roles and responsibilities

1. Standard User

Responsibilities:

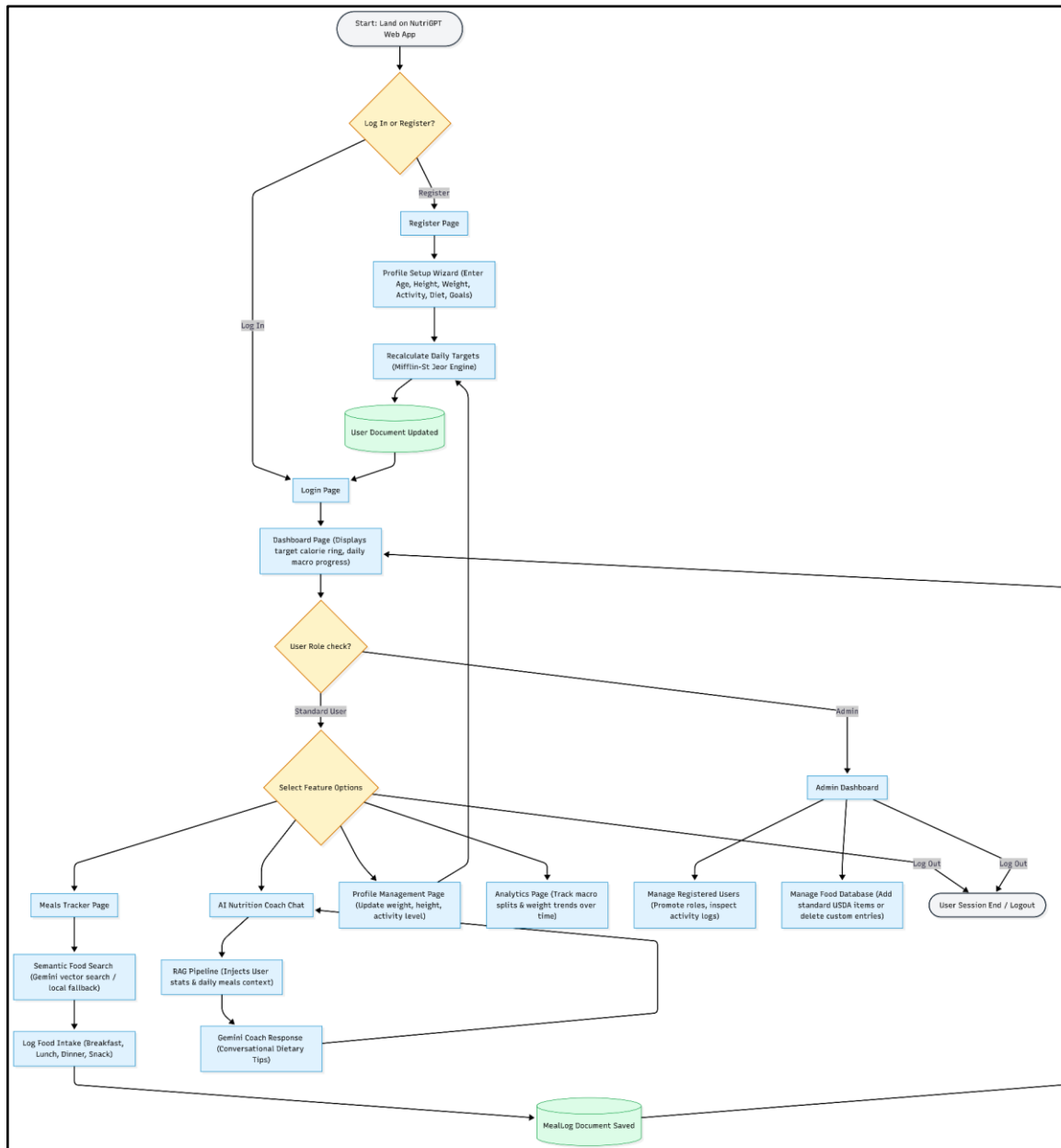
- **Account Management:** Register, log in, modify password, complete profile setup wizard. **Tracker Interaction:** Log foods under breakfast, lunch, dinner, snack, and create custom foods.
- **AI Coaching:** Query the AI coach for meals assessment, macro summaries, and tips.
- **Analytics:** Review dashboard charts to monitor weight changes and calorie logs.

2. Administrator

Responsibilities:

- **User Oversight:** View list of registered accounts and promote standard users to admin.
- **Food Database Management:** Create, edit, and delete items in the food database, and trigger database seed scripts.
- **Platform Monitoring:** Toggle production vector search models and check server status logs.

User Flow



1. Standard User Journey

- **Landing & Authentication:** Accesses the application, registers, or logs in.
- **Profile Setup:** If new, completes the profile form. Daily calorie and macro targets are automatically calculated.
- **Dashboard:** Redirects to the homepage showing targets, calorie wheels, and logged intake.

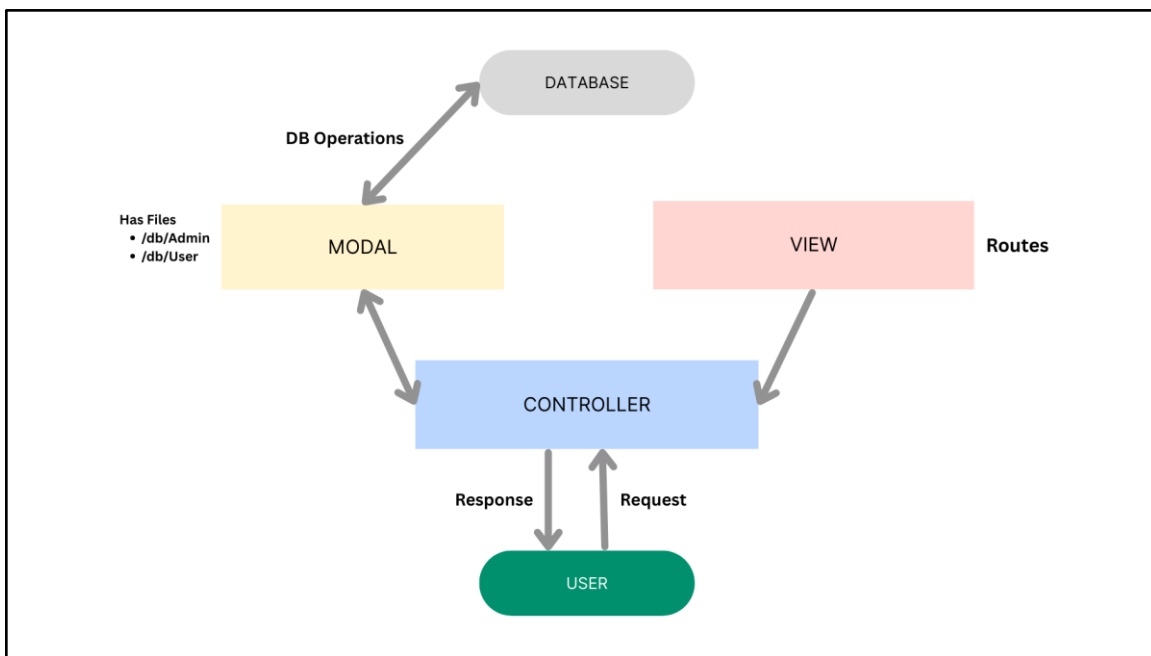
- **Meal Tracking:** Navigates to `/meals` or `/foods`. Performs semantic searches, sets portion quantities, and logs meals.
- **AI Consultation:** Opens `/chat` and speaks to the nutrition coach. The RAG pipeline loads profile details and meal history to answer context-aware queries.

Progress Monitoring: Reviews charts for target adherence.

2. Administrator Journey

- **Login:** Administrator logs in.
- **Navigation:** Navigates to `/admin` using the admin route link in the navigation header.
- **Database Maintenance:** Views and filters user metrics, adds new USDA reference foods, or deletes custom items.

MVC Pattern



3

The NutriGPT backend is structured using the Model-View-Controller (MVC) architectural pattern:

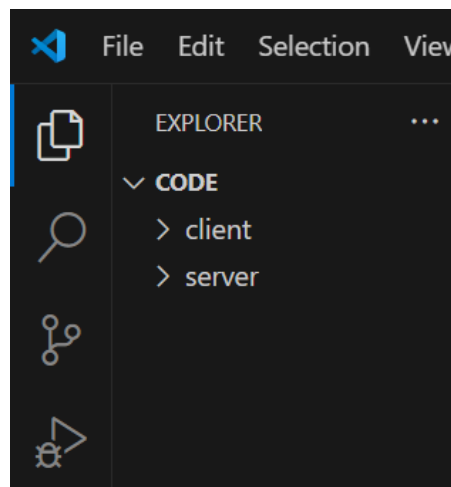
- **Model Layer (Data):** Structured schemas (Mongoose) that represent and save data logic inside MongoDB. Contains built-in helper hooks (e.g., hash passwords, calculate calorie targets). Located in models.

- **Controller Layer (Business Logic):** Functions invoked by incoming requests. Validates body inputs, invokes Mongoose commands, processes AI embedding queries, and returns JSON responses. Located in controllers.
- **View Layer (Client Presentation):** In the full-stack layout, the View is decoupled as the React client workspace located in client which communicates with the backend via API endpoints. The backend routers located in routes direct endpoints to their respective controllers.

Project Setup And Configuration.

Creating a project folder

- Extract or clone the main project directory.
- Navigate to the project root folder.
- You will see the two main folders: `client` and `server`.
- Open this project directory in Visual Studio Code.



Client setup (installing React app)

1. Open a terminal in VS Code and navigate to the `client` folder:

```
`cd client`
```

2. Install dependencies:

```
`npm install`
```

3. Start the local React client:

```
`npm run dev`
```

Server setup

1. Open a separate terminal tab in VS Code and navigate to the `server` folder:

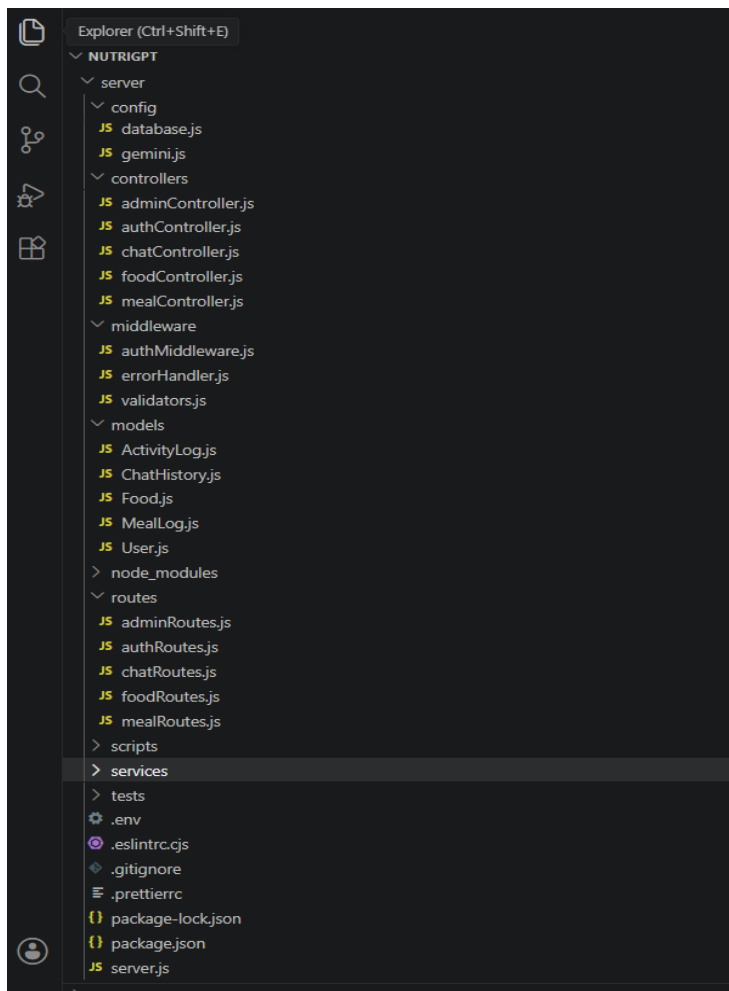
```
`cd server`
```

2. Install dependencies:

```
`npm install`
```

Backend Development:

Backend Structure:



Development:

config/: Configures database and third-party API clients.

- **database.js**: Establishes MongoDB connection using Mongoose.
- **gemini.js**: Initializes the Google Gemini AI client.
-

controllers/: Handles the request-response cycle and executes backend logic.

- **authController.js**: Handles user signup, login, profile registration, and token validation.
- **foodController.js**: Handles food creation, semantic and textual searches, and custom user food lists.
- **mealController.js**: Handles logging, editing, deleting daily meal logs, and totals tracking.
- **chatController.js**: Processes RAG-based chat prompts and reads/writes multi-turn chat sessions.
- **adminController.js**: Contains operations to fetch all users and update roles.

middleware/: Request processing middleware.

- **authMiddleware.js**: Validates incoming JWT tokens, attaches the user object, and verifies admin roles.
- **errorHandler.js**: Global error catcher formatting exception responses.
- **validators.js**: Validates request payloads for profiles, auth registers, and meal inputs.

models/: Mongoose schemas defining MongoDB collections.

- **User.js**: Schema for user credentials, health profile parameters, and targets.
- **Food.js**: Schema for food nutrition, diet tags, and vector embeddings.
- **MealLog.js**: Schema for meal logs and daily totals.
- **ChatHistory.js**: Schema for conversational chat message arrays.
- **ActivityLog.js**: Audit schema tracking user events.

routes/: Routes HTTP endpoints to controllers.

- [authRoutes.js](#): /routes/authRoutes.js
- [foodRoutes.js](#): routes/foodRoutes.js)
- [mealRoutes.js](#) : /routes/mealRoutes.js)
- [chatRoutes.js](#) : /routes/chatRoutes.js)
- [adminRoutes.js](#) : /routes/adminRoutes.js)

services/: Encapsulates service integrations.

- **ragService.js**: Generates RAG prompts by gathering daily foods and physical statistics.
- **embeddingService.js**: Interfaces with the Gemini embedding model to generate numerical vectors.
- **vectorSearchService.js**: Handles local cosine-similarity comparisons or MongoDB Atlas Vector Search pipelines.

scripts/: Utility tools.

- **seedFoods.js**: Populates the food collection with initial USDA references.
- **generateEmbeddings.js**: Generates embeddings for seeded foods lacking them.

Entry Point – server.js

The main file that boots the backend server:

- Initializes express and configures security headers (helmet) and CORS rules.
- Connects to the database by executing `connectDB()` from `config/database.js`.
- Configures logging middleware (morgan) and rate-limiting blocks.
- Mounts routes under `/api/auth`, `/api/foods`, `/api/meals`, `/api/chat`, and `/api/admin`.
- Starts the HTTP server on the configured port.

Database Development:

1\ Configure MongoDB:

Install Mongoose:

```
`npm install mongoose`
```

Create Database connection:

```
const mongoose = require("mongoose");

const connectDB = () => {
  mongoose
    .connect(process.env.MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    })
    .then(() => {
      console.log("Connected to MongoDB");
    })
    .catch((err) => {
      throw new Error(`Could not connect to MongoDB: ${err}`);
    });
};

module.exports = connectDB;
```

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below.

1. `const mongoose = require('mongoose');`

- Purpose: Loads the Mongoose library so you can use it in your application.
- Mongoose is a powerful Object Data Modeling (ODM) tool that helps interact with MongoDB using JavaScript objects.

2. `mongoose.connect(...)`

- `mongodb://127.0.0.1:27017/EventManager'`
- `mongodb://`: Specifies the MongoDB protocol.
- `127.0.0.1`: [Localhost](#) IP address – means the database is running on your local machine.
- `27017`: Default port for MongoDB.
- Roomly: Name of the database you want to use.
 - If this database doesn't exist, MongoDB will create it once data is inserted.

`useNewUrlParser: true`: Uses MongoDB's updated URL string parser.

`useUnifiedTopology: true`: Enables the new connection engine, which improves connection stability and performance.

3. `..then(()=> console.log('Connected to MongoDB'))`

- If the connection is successful, this code runs.

- It prints a confirmation message to your console:

```
4..catch((err)=>console.error('MongoDB connection error:', err));
```

- If the connection fails, the error is caught here.
- Helps with debugging by printing the error details

...

Create Schemas:

Configure Mongoose schemas with indexes, models, and calculation hooks:

User Schema: Contains properties for name, email, role, health profile parameters, and methods to compute daily target calorie and macronutrient limits.

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
      maxlength: [80, 'Name cannot exceed 80 characters'],
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email address'],
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: [6, 'Password must be at least 6 characters'],
      select: false,
    },
    role: {
      type: String,
      enum: ['user', 'admin'],
      default: 'user',
    },
    profile: {
      age: { type: Number, min: 10, max: 120 },
      weight: { type: Number, min: 20, max: 400 }, // kg
      height: { type: Number, min: 50, max: 260 }, // cm
      gender: { type: String, enum: ['male', 'female', 'other'] },
      activityLevel: {
        type: String,
        enum: ['sedentary', 'light', 'moderate', 'active', 'very_active'],
        default: 'moderate',
      },
      goal: {
        type: String,
        enum: ['weight_loss', 'muscle_gain', 'maintenance'],
        default: 'maintenance',
      },
      diet: {
        type: String,
        enum: ['none', 'vegetarian', 'vegan', 'keto', 'paleo'],
        default: 'none',
      },
      allergies: [{ type: String, trim: true }],
    },
    nutritionTargets: {
      calories: { type: Number, default: 2000 },
      protein: { type: Number, default: 100 }, // grams
      carbs: { type: Number, default: 250 },
```

```

    },
    nutritionTargets: {
      calories: { type: Number, default: 2000 },
      protein: { type: Number, default: 100 }, // grams
      carbs: { type: Number, default: 250 },
      fat: { type: Number, default: 65 },
    },
    isProfileComplete: { type: Boolean, default: false },
    isActive: { type: Boolean, default: true },
    lastLoginAt: { type: Date },
    avatar: { type: String, default: null }, // base64 data URL, compressed client-side
    bio: { type: String, maxLength: 200, default: '' },
  },
  { timestamps: true }
);

userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

userSchema.methods.comparePassword = async function (enteredPassword) {
  return bcrypt.compare(enteredPassword, this.password);
};

// Mifflin-St Jeor equation for BMR, then TDEE, then goal-adjusted calorie target
userSchema.methods.calculateNutritionTargets = function () {
  const { age, weight, height, gender, activityLevel, goal } = this.profile;
  if (!age || !weight || !height) return null;

  let bmr;
  if (gender === 'female') {
    bmr = 10 * weight + 6.25 * height - 5 * age - 161;
  } else {
    bmr = 10 * weight + 6.25 * height - 5 * age + 5;
  }

  const activityMultipliers = {
    sedentary: 1.2,
    light: 1.375,
    moderate: 1.55,
    active: 1.725,
    very_active: 1.9,
  };

  const tdee = bmr * (activityMultipliers[activityLevel] || 1.55);

  let targetCalories = tdee;
  if (goal === 'weight_loss') targetCalories = tdee - 500;
  if (goal === 'muscle_gain') targetCalories = tdee + 300;

  targetCalories = Math.round(targetCalories);

  // Macro split: protein 30% / carbs 40% / fat 30% (adjustable per goal)
  const protein = Math.round((targetCalories * 0.3) / 4);
  const carbs = Math.round((targetCalories * 0.4) / 4);
  const fat = Math.round((targetCalories * 0.3) / 9);

  return { calories: targetCalories, protein, carbs, fat };
};

module.exports = mongoose.model('User', userSchema);

```



Food Schema: Stores ingredient macros and a numeric array for vector embeddings.

```
const mongoose = require('mongoose');

const foodSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
      index: true,
    },
    category: {
      type: String,
      trim: true,
      index: true,
    },
    servingSize: {
      amount: { type: Number, default: 100 },
      unit: { type: String, default: 'g' },
    },
    // All values per serving size above
    calories: { type: Number, required: true },
    protein: { type: Number, required: true }, // grams
    carbs: { type: Number, required: true },
    fat: { type: Number, required: true },
    fiber: { type: Number, default: 0 },
    sugar: { type: Number, default: 0 },
    sodium: { type: Number, default: 0 }, // mg

    dietTags: [{
      type: String,
      enum: ['vegetarian', 'vegan', 'keto', 'paleo', 'gluten_free', 'dairy_free'],
    }],

    // Text used to generate the embedding (name + category + macros summary)
    embeddingText: { type: String },

    // Vector embedding for semantic search (1536 dims for text-embedding-3-small)
    embedding: {
      type: [Number],
      default: undefined,
      select: false, // don't return huge arrays by default
    },

    source: {
      type: String,
      enum: ['usda', 'manual', 'admin', 'user'],
      default: 'manual',
    },

    // Quick-add foods created by a regular user for their own logging.
    // These don't show up in shared semantic search results for other users.
    isCustom: { type: Boolean, default: false },
    createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  },
  { timestamps: true }
);

foodSchema.index({ name: 'text', category: 'text' });

module.exports = mongoose.model('Food', foodSchema);
```



MealLog Schema: Stores logged items and daily totals.

```
const mongoose = require('mongoose');

const mealItemSchema = new mongoose.Schema(
  {
    food: { type: mongoose.Schema.Types.ObjectId, ref: 'Food', required: true },
    name: { type: String, required: true }, // denormalized for fast read
    quantity: { type: Number, default: 1 }, // multiplier of serving size
    calories: { type: Number, required: true },
    protein: { type: Number, required: true },
    carbs: { type: Number, required: true },
    fat: { type: Number, required: true },
  },
  { _id: true }
);

const mealLogSchema = new mongoose.Schema(
  {
    user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, index: true },
    date: { type: String, required: true, index: true }, // YYYY-MM-DD for easy daily grouping
    mealType: {
      type: String,
      enum: ['breakfast', 'lunch', 'dinner', 'snack'],
      required: true,
    },
    items: [mealItemSchema],
    totals: {
      calories: { type: Number, default: 0 },
      protein: { type: Number, default: 0 },
      carbs: { type: Number, default: 0 },
      fat: { type: Number, default: 0 },
    },
  },
  { timestamps: true }
);

// Recalculate totals whenever items change
mealLogSchema.pre('save', function (next) {
  this.totals = this.items.reduce(
    (acc, item) => ({
      calories: acc.calories + item.calories * item.quantity,
      protein: acc.protein + item.protein * item.quantity,
      carbs: acc.carbs + item.carbs * item.quantity,
      fat: acc.fat + item.fat * item.quantity,
    }),
    { calories: 0, protein: 0, carbs: 0, fat: 0 }
  );
  next();
});

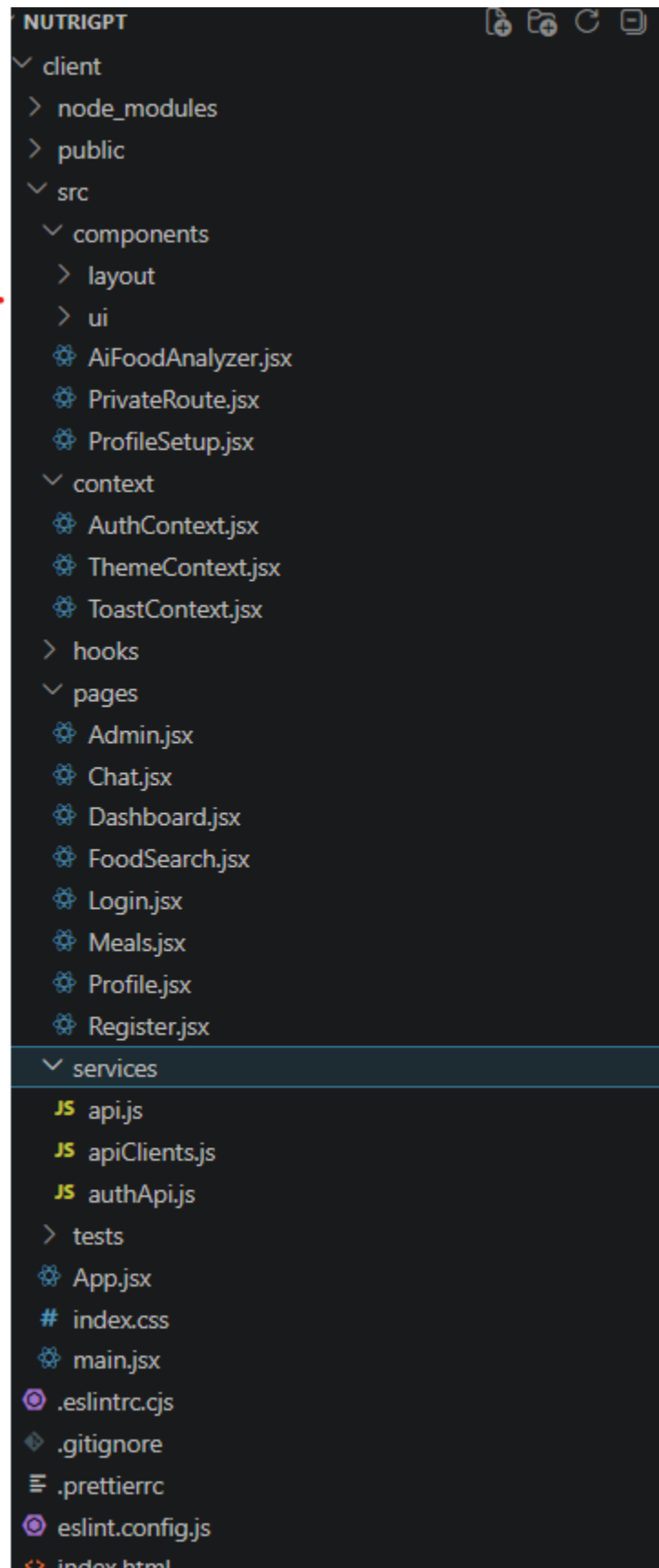
mealLogSchema.index({ user: 1, date: 1 });

module.exports = mongoose.model('MealLog', mealLogSchema);
```



Front-End Development:

Structure:



The frontend utilizes a component-based layout:

src/components/layout/

- **AppShell.jsx:** Renders the main dashboard shell layout. Contains the navigation header, user profile dropdowns, and theme toggle buttons.

src/components/ui/

- **index.jsx:** Contains reusable UI elements like buttons, inputs, dialog modals, cards, and select lists.

src/components/

- **AiFoodAnalyzer.jsx:** Sub-component enabling users to type natural-language food descriptions and receive nutritional breakdowns.
- **PrivateRoute.jsx:** Router wrapper validating session status.
- **ProfileSetup.jsx:** Wizard container prompting new users for stats upon registration.

src/pages/

- **Dashboard.jsx:** The home dashboard page. Renders remaining calorie rings and macro charts using Recharts.
- **Meals.jsx:** Renders logged food item lists and forms to input meals.
- **FoodSearch.jsx:** Semantic search interface displaying food details and nutrition targets.
- **Chat.jsx:** Session chat panel with the RAG nutrition coach.
- **Profile.jsx:** Displays user statistics and enables target recalculations.
- **Admin.jsx:** Admin dashboard displaying user list controls.
- **Login.jsx & Register.jsx:** Authentication pages.

Project Execution:

Step 1: Set Up the Frontend (React App)

1. Open a terminal and navigate to the client folder:

```
`cd client`
```

2. Start the Vite React client:

```
`npm run dev`
```

3. The development server will run on:

```
`http://localhost:5173`
```

Step 2: Set Up the Backend (Express Server)

1. Open a separate terminal and navigate to the server folder:

```
`cd server`
```

2. Install dependencies if not already done:

```
`npm install`
```

Step 3: Configure Environment Variables

1. In the `server` folder, create a file named `.env`:

```
```env
```

- PORT=5500
- MONGO\_URI=mongodb://localhost:27017/NutriGPT
- JWT\_SECRET=nutrigpt\_super\_secret\_key\_change\_this
- JWT\_EXPIRE=1d
- NODE\_ENV=development
- GEMINI\_API\_KEY=your\_gemini\_api\_key\_here
- GEMINI\_CHAT\_MODEL=gemini-2.5-flash
- GEMINI\_EMBEDDING\_MODEL=gemini-embedding-001
- RATE\_LIMIT\_WINDOW\_MS=900000
- RATE\_LIMIT\_MAX=100
- AI\_RATE\_LIMIT\_MAX=20
- USE\_ATLAS\_VECTOR\_SEARCH=false

```
```
```

Step 4: Seed Database and Generate Embeddings

Seeding initializes the food items database and generates vector embeddings:

```
`npm run seed:foods`
```

```
`npm run seed:embeddings`
```

Step 5: Start the Backend Server

Start the Express backend:

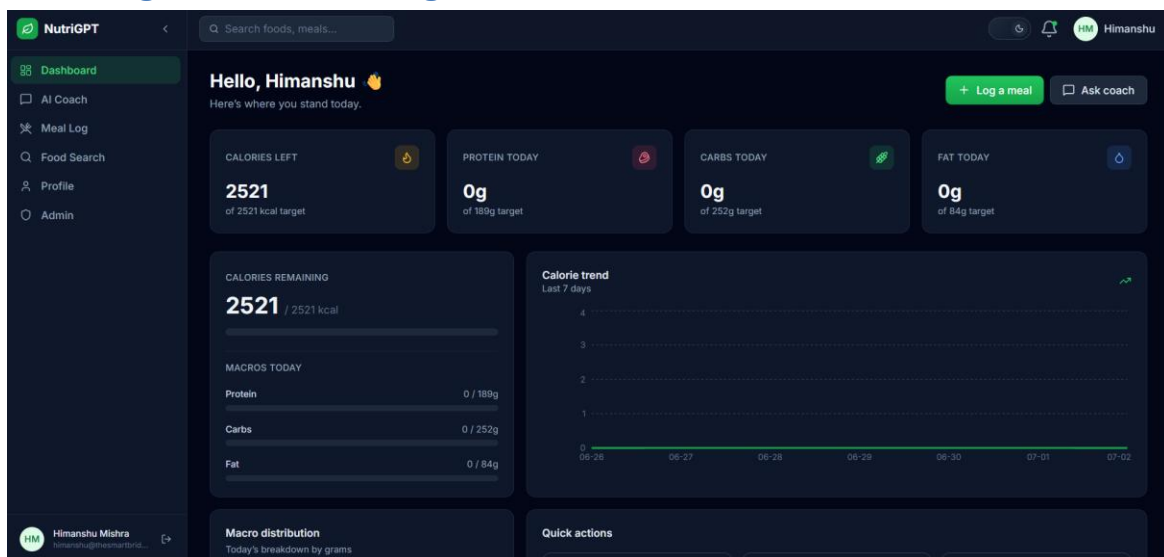
```
`npm run dev`
```

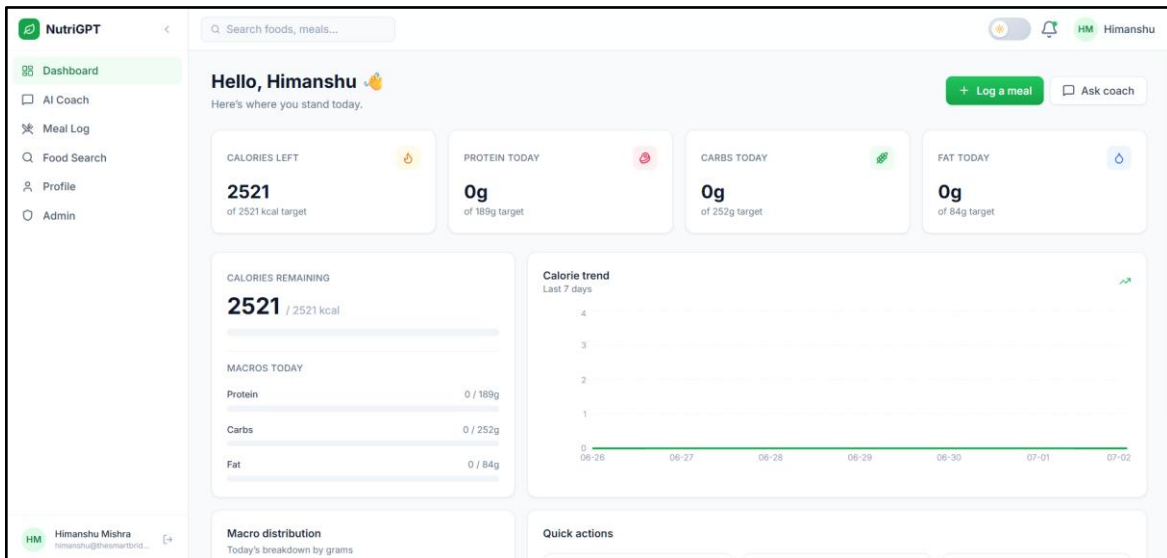
The server will run on:

```
`http://localhost:5500`
```

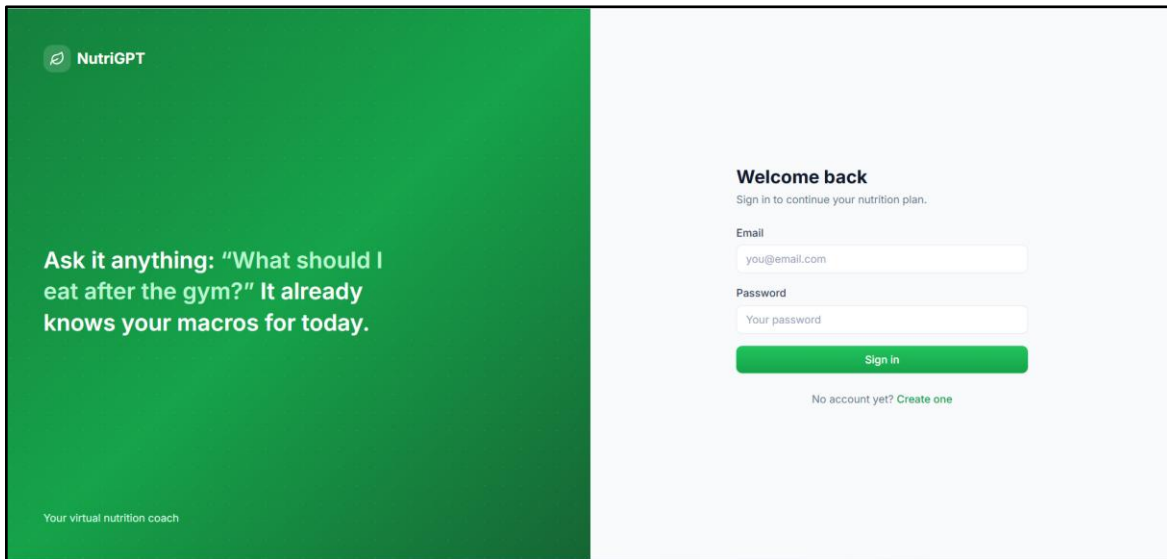
Output ScreenShots:

HomePage / Dashboard Page:





Login Page:



NutriGPT

Ask it anything: "What should I eat after the gym?" It already knows your macros for today.

Your virtual nutrition coach

Welcome back
Sign in to continue your nutrition plan.

Email
you@email.com

Password
Your password

[Sign in](#)

No account yet? [Create one](#)

Register Page:



NutriGPT

Nutrition advice that knows
your body, your goals, and
what's actually in your fridge.

Powered by retrieval-augmented AI

Create your account

Start your personalized nutrition journey today.

Full name

Email

Password

Create account

Already have an account? [Sign in](#)

Profile Setup Page:



NutriGPT

Search foods, meals...

Dashboard

AI Coach

Meal Log

Food Search

Profile

Admin

Profile

Manage your account, health profile, and password.



Himanshu Mishra
himanshu@thesmartbridge.com

DAILY TARGETS

| | |
|----------|-----------|
| Calories | 2521 kcal |
| Protein | 189 g |
| Carbs | 252 g |
| Fat | 84 g |

Member since June 2026

Account

Health profile

Password

Account details

Update your name, email address, and short bio.

Full name

Email address

Short bio (optional - max 200 chars)

0/200

Save account details

Meals Tracker Page:



Food Search Page:

Q Search foods, meals...

Meal log 02-07-2026

Track what you eat, day by day.

600 kcal / 2521 19g protein 88g carbs 18g fat

Describe what you ate
Type anything — "2 roti", "1 plate biryani", "coffee with milk". The AI figures out the macros.

Lunch e.g., 2 roti and 1 bowl dal makhani [Analyze](#)

Try: 2 roti and 1 bowl dal makhani 1 plate biryani with raita poha with chutney 3 idli with sambar and chutney grilled chicken sandwich

Or search the food database

Add a meal

breakfast Search a food to add... [Find](#)

Can't find it? [Quick add a custom food](#)

lunch [Remove](#)

Paratha × 1 200 kcal

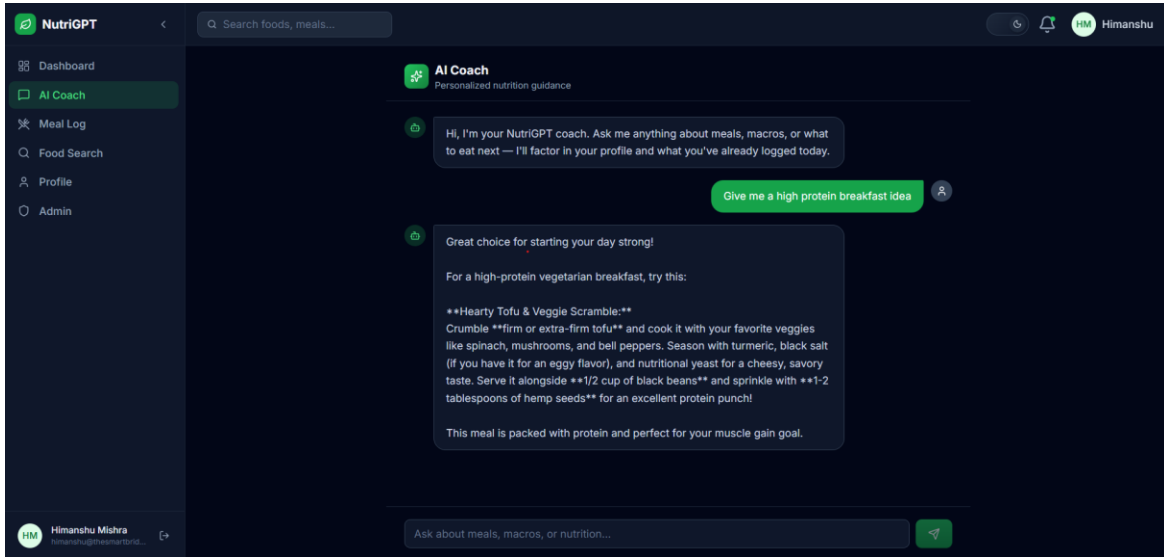
200 kcal total

breakfast [Remove](#)

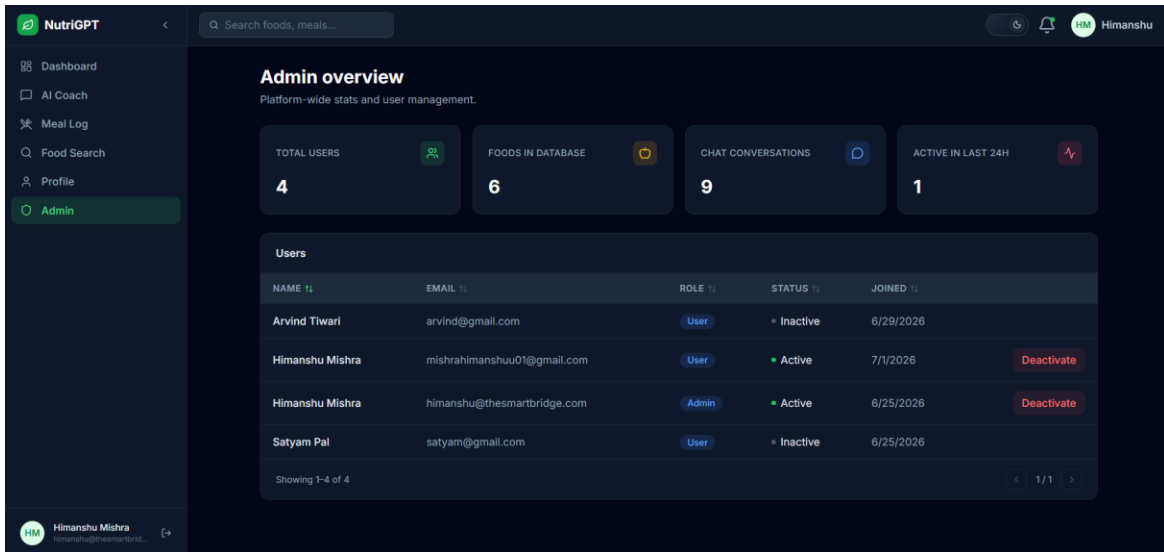
Biryani × 1 400 kcal

400 kcal total

AI Coach Chat Page:



Admin Dashboard Page:



Demo video link

Code drive link

https://drive.google.com/drive/folders/12So725-hcZA-uJZZCmFiJHnZGz1_sfC?usp=drive_link